class won't be bound to anything and will have only default values in its properties.

The binding of the `T` options class to `ConfigurationSection` happens when you first request `IOptions<T>.` The object is registered in the DI container as a singleton, so it's bound only once.

This setup has one catch: you can't use the `reloadOnChange` parameter I described in section 10.2.4 to reload your strongly typed options classes when using `IOptions<T>`. `IConfiguration` will still be reloaded if you edit your appsettings.json files, but it won't propagate to your options class.

If that fact seems like a step backward or even a deal-breaker, don't worry. `IOptions<T>` has a cousin, `IOptionsSnapshot<T>`, for such an occasion.

10.3.2 Reloading strongly typed options with IOptionsSnapshot

In section 10.3.1, you used `IOptions<T>` to provide strongly typed access to configuration. Using `IOptions<T>` to provide strongly typed access to configuration provided a nice encapsulation of the settings for a particular service, but with a specific drawback: the options class never changes, even if you modify the underlying configuration file from which it was loaded, such as appsettings.json.

This situation isn't always a problem (you generally shouldn't be modifying files on live production servers anyway), but if you need this functionality, you can use the `IOptionsSnapshot<T>` interface. Conceptually, `IOptionsSnaphot<T>` is identical to `IOptions<T>` in that it's a strongly typed representation of a section of configuration. The difference is when and how often the POCO options objects are created when they're used: